

Z5008 Big Data Lab

Course Project Guidelines

IIT Madras Zanzibar · Dr. Innocent Nyalala · Even Semester 2026

The Project: Real-Time Lakehouse + MLOps Intelligence Platform (50% of grade)

Build a production-grade, end-to-end Big Data system that ingests real-time data, stores it in a Lakehouse, processes it with Spark, trains an ML model, deploys it as an API, and monitors it with dashboards.

Groups of **2 students**. Choose any real-world domain that excites your team. You will demo a *live, running system* in Week 16.

Project Overview

What You Will Build

Your system must implement the following 8-layer architecture:

8-Layer Architecture Checklist

Layer	Component	Minimum expectation
1	Data ingestion	Kafka producer streaming real-time data (not only batch).
2	Object storage	MinIO (S3-compatible) using Delta Lake <i>or</i> Apache Iceberg tables.
3	Processing	Apache Spark (batch + Structured Streaming).
4	Orchestration	Scheduled pipeline in Airflow <i>or</i> Dagster.
5	Model training	Spark MLlib <i>or</i> Ray trained on your lakehouse data.
6	Experiment tracking	MLflow runs + model registry (staging/production).
7	Model serving	BentoML REST API in Docker (accepting POST requests).
8	Monitoring	Grafana dashboard with meaningful real-time system metrics.

Suggested Domains

- Agriculture (crop yields, sensor data)
- Healthcare (patient records, clinical)
- Finance (transaction fraud, pricing)
- Transportation (fleet, traffic, routing)
- Environment (air quality, weather)
- Retail (sales, inventory, demand)
- Energy (smart grid, consumption)
- Social media (sentiment, trends)
- Sports analytics

Tip

Choose a domain where you can find or generate a **realistic streaming dataset**. Open datasets: Kaggle, UCI ML Repository, government open data portals, or simulate your own with a Python data generator.

Milestone Timeline

Week	Date	Milestone	Deliverable
2	16 Mar	Project Brief Released	Read brief; discuss domain ideas with team
3	23 Mar	Team Formation	Submit team sheet on Classroom (names + domain idea)
4	30 Mar	Proposal Due	2-page PDF: domain, architecture diagram, dataset, tool stack
8	27 Apr	Progress Review 1	10-min live demo: data flowing end-to-end (ingestion + storage)
12	25 May	Progress Review 2	15-min demo: Spark processing + ML training working
15	15 Jun	Code Freeze	Final GitHub commit; no major changes after this
16	22 Jun	Final Presentation	20-min live demo + Q&A + submit 8-min video

Deliverables in Detail

Deliverable 1: Project Proposal (Due Week 4 — 30 March)

Submitted as a 2-page PDF on Moodle. Must include:

1. **Team members** (names, roll numbers)
2. **Problem statement:** What problem are you solving? Who benefits?
3. **Domain and dataset:** What data will you use? How will you stream it?
4. **System architecture diagram:** All 8 layers shown with actual tool names
5. **Technology stack:** Specific tools chosen for each layer (justify choices)
6. **Novelty:** What is interesting or unique about your project?
7. **Risk assessment:** What could go wrong? How will you mitigate it?

Graded: Pass/Fail (must pass to continue; revision allowed once)

Deliverable 2: GitHub Repository (Ongoing — Graded at Week 15)

Your GitHub repo must contain:

- README.md with clear setup and run instructions (anyone should be able to run it)
- docker-compose.yml for all services
- notebooks/ — all Jupyter notebooks with clear markdown explanations
- spark_jobs/ — production Spark batch and streaming jobs (Python files, not just notebooks)
- dags/ — Airflow or Dagster pipeline definitions
- ml/ — training scripts, MLflow experiment code
- api/ — BentoML service definition and Dockerfile
- dashboards/ — Grafana dashboard JSON exports
- data/ — sample data or data generation scripts
- Regular commits (must show team contribution; at least weekly commits)
- .env.example with all required variables documented

Deliverable 3: Final Presentation + Live Demo (Week 16 — 22 June)**20-minute live demonstration + 8-10 minute Q&A:**

- All 8 system layers must be **live and running** on your laptop
- Demonstrate data flowing from **Kafka** → **MinIO** → **Spark** → **ML model** → **API**
- Show **MLflow experiments** with at least 3 model runs compared
- Call your **deployed API** with a real data point and show prediction
- Show **Grafana dashboard** with live metrics
- Run **10× load test** (demonstrate system stability under load)
- Submit an **8-minute video walkthrough** (for remote evaluation)

Grading Rubric (50% of course grade)**Full Rubric Breakdown**

Component	Marks	Criteria
Proposal	10	Problem clarity, architecture quality, tech choices, feasibility
Progress Reviews (×2)	10	Technical progress, team coordination, ability to demo
GitHub Repository	10	Code quality, documentation, commit history, reproducibility
Presentation + Video	20	All layers working, live demo, 10× load stability, MLOps maturity, Clarity, technical depth, Q&A handling, 8-min video quality
Total	50	

Bonus Points (+5%)

Add a **natural-language query interface** to your data:

- Use any LLM (GPT-4o, Gemini, local Llama via Ollama, etc.)
- Users type a question in plain English; system translates to Spark SQL and returns results
- Must work live in the demo
- Implement safety: SQL injection prevention, read-only access

This is for the truly ambitious team. It is not required and will not penalise teams who skip it.

Technical Requirements

Mandatory Requirements (All 8 Layers)

Requirement	Minimum expectation
Streaming ingestion	Real data must flow through Kafka (not just batch loading).
Lakehouse storage	Data stored in Delta Lake <i>or</i> Apache Iceberg on MinIO.
Spark processing	At least one batch job <i>and</i> one streaming job.
Orchestration	At least one scheduled Airflow DAG or Dagster job.
ML model	Trained using Spark MLlib or Ray on your lakehouse data.
MLflow	At least 5 tracked experiment runs; model registered in the registry.
Deployed API	BentoML REST endpoint in Docker, accepting POST requests.
Monitoring	At least 3 Grafana panels showing meaningful system metrics.

Code Quality Standards

- No hardcoded credentials in code (use environment variables / `.env`).
- Production Spark jobs must be `.py` files, not only notebooks.
- All services run via `docker-compose up -d`.
- README enables a stranger to run your entire system from scratch.
- Unit tests for at least 2 non-trivial functions.

Team Policies

Team Formation

- Teams of 2 students (no exceptions).
- Choose your own teams; unformed by Week 3 = assigned.
- Mix skills (e.g., one strong in Python, one strong in data).
- Register on the classroom by end of Week 3.

Contribution Tracking

- GitHub commit history is reviewed.
- All team members must commit code.
- Peer evaluation at end of semester ($\pm 10\%$ individual adjustment).
- Free riders will be penalised; exceptional contributors rewarded.

Academic Integrity

- You may use AI coding tools (GitHub Copilot, ChatGPT) but must **declare their use** in your README
- Any AI-assisted code must be understood and explainable during Q&A
- Copying another team's work = zero for **all team members**
- Using open-source code is allowed; **always cite your sources** in comments

Resources and Support

Datasets (Starting Points)

- Kaggle: <https://kaggle.com/datasets>
- UCI ML Repository: <https://archive.ics.uci.edu>
- Our World in Data: <https://ourworldindata.org>
- Government open data portals
- IoT simulation: Python `Faker` library

Reference Architecture Examples

- Databricks End-to-End ML Demo
- Netflix Metacat (data governance)
- Uber Michelangelo (MLOps)
- LinkedIn DataHub (data catalog)
- Airbnb Minerva (metrics platform)